FPLI Mandatory Assignment

Functional Programming & Language Implementations — Spring 2026

Part A · Compiler **Part B · Dynamic Type Checker**

Roskilde University · Morten Rhiger · 2026

Constants, Arithmetic & Comparisons

- true/false compiled as IPUSH 1 / IPUSH 0
- Arithmetic: + − * / % all follow the same pattern:
  - compile left, extend env with placeholder, compile right, emit op
- Unary negation: no NEG instruction in VM ⟶
  - compiled as 0 − e using IPUSH 0 then ISUB
- EQ uses IEQ directly

```
| Syntax.BOOL b ->
    [Asm.IPUSH (if b then 1 else 0)]

| Syntax.ADD (e1, e2) ->
    comp env e1 @
    comp ("" :: env) e2 @
    [Asm.IADD]


| Syntax.NEG e ->
    [Asm.IPUSH 0] @
    comp ("" :: env) e @
    [Asm.ISUB]
```

Comparison Operator Workarounds

The VM only has IEQ and ILT — everything else are constructed from these.

!= Compute ==, then check == 0

```
| Syntax.NEQ (e1, e2) ->
    comp env e1 @
    comp ("" :: env) e2 @
    [Asm.IEQ] @
    [Asm.IPUSH 0] @
    [Asm.IEQ]   // equal-to-zero = not equal
```

<= $a \leq b \equiv \text{NOT}\ (b < a)$

```
| Syntax.LTE (e1, e2) ->
    comp env e2 @       // reversed!
    comp ("" :: env) e1 @
    [Asm.ILT] @
    [Asm.IPUSH 0] @
    [Asm.IEQ]
```

> $a > b \equiv b < a$

```
| Syntax.GT (e1, e2) ->
    comp env e2 @
    comp ("" :: env) e1 @
    [Asm.ILT]
```

>= $a \geq b \equiv \text{NOT}\ (a < b)$

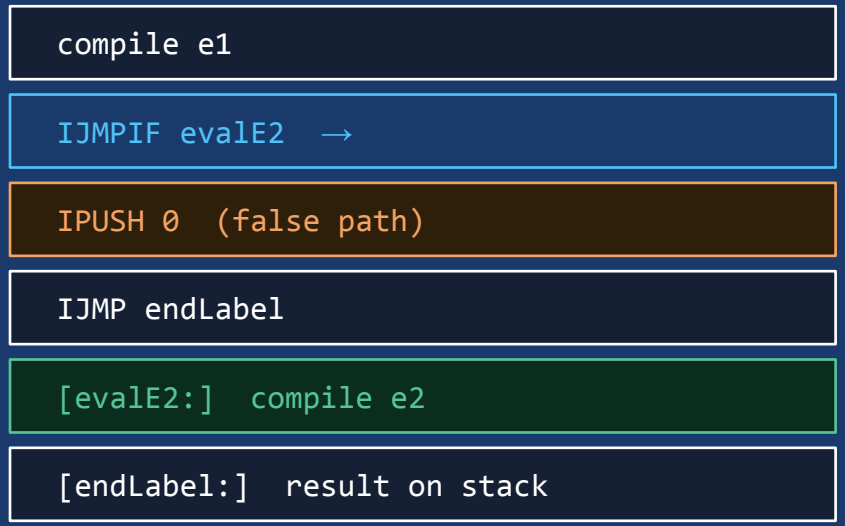
```
| Syntax.GTE (e1, e2) ->
    comp env e1 @
    comp ("" :: env) e2 @
    [Asm.ILT] @
    [Asm.IPUSH 0] @
    [Asm.IEQ]
```

Part A ·

Short-Circuit && and ||

e1 && e2 must NOT evaluate e2 if e1 is false — implemented with conditional jumps.

AND logic flow



OR — symmetric:

```
| OR (e1, e2) ->
  comp env e1 @
  [IJMPIF trueLabel] @
  comp env e2 @          // only if e1 false
  [IJMP endLabel] @
  [ILAB trueLabel] @
  [IPUSH 1] @            // short-circuit true
  [ILAB endLabel]
```

Part A ·

Multi-Argument Functions — Calling Side

Arguments pushed left-to-right. After return, popArgs cleans up with ISWAP + IPOP × n.

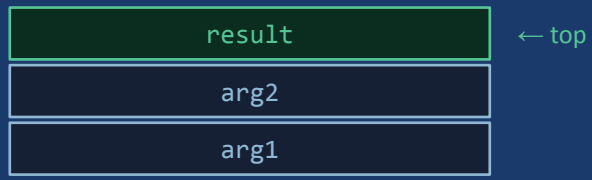
```
| Syntax.CALL (f, es) ->
    compArgs env es @
    [Asm.ICALL f] @
    popArgs (List.length es)


and compArgs env = function
    | [] -> []
    | e :: es ->
        comp env e @
        compArgs ("" :: env) es
```

popArgs — cleanup after return:

```
let rec popArgs n =
  if n = 0 then []
  else
    [Asm.ISWAP] @  // result past arg
    [Asm.IPOP]  @  // remove arg
    popArgs (n - 1)
```

Stack after f(a, b) returns:

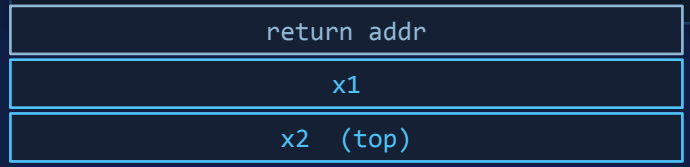


Part A ·

Multi-Argument Functions — Definition Side

```
| ((f, (xs, e)) :: funcs, e1) ->
    compProg (funcs, e1) @
    [Asm.ILAB f] @
    comp ("" :: List.rev xs) e @
    [Asm.ISWAP] @
    [Asm.IRETN]


// "" = placeholder for return address
// List.rev xs — leftmost arg lowest
```



Return address

ICALL pushes it below the args. The "" placeholder accounts for it in varpos.

List.rev xs

Caller pushes args left-to-right. We reverse so x1 has offset 0 from the return addr.

ISWAP + IRETN

Result is on top. ISWAP puts it below return addr, IRETN pops addr and jumps.

Part B

Dynamic Type Checker — Design

Types are checked at runtime. The interpreter distinguishes IntVal from BoolVal and reports meaningful errors.

```
type value =
  | IntVal  of int
  | BoolVal of bool

let expectInt v =
  match v with
  | IntVal i  -> i
  | BoolVal _ ->
    failwith "expected int, got bool"

let expectBool v =
  match v with
  | BoolVal b -> b
  | IntVal _  ->
    failwith "expected bool, got int"
```

Using the guards:

```
let rec eval env e =
    match e with
    | INT i -> IntVal i
    | BOOL b -> BoolVal b

    | VAR x -> lookup x env

    | ADD (e1, e2) ->
    IntVal (
      expectInt (eval env e1)
    + expectInt (eval env e2))

   | LT (e1, e2) ->
    BoolVal (
      expectInt (eval env e1)
    < expectInt (eval env e2))
```

Part B

Polymorphic == / != and Short-Circuit

== and != — polymorphic but not mixed:

```
| EQ (e1, e2) ->
    let v1 = eval env e1
    let v2 = eval env e2
    match v1, v2 with
    | IntVal i1,  IntVal i2  -> BoolVal (i1 = i2)
    | BoolVal b1, BoolVal b2 -> BoolVal (b1 = b2)
    | _ -> failwith "== requires two ints OR two bools"
```

Short-circuit AND / OR — natural in the interpreter:

```
| AND (e1, e2) ->
    let b1 = expectBool (eval env e1)
    if b1
    then BoolVal (expectBool (eval env e2))
    else BoolVal false
    // e2 never called if e1 = false
```

```
| OR (e1, e2) ->
    let b1 = expectBool (eval env e1)
    if b1 then BoolVal true
    else BoolVal (expectBool (eval env e2))

    // e2 never called if e1 = true
```

DEMO

Compiler in Action

Demo - Tracing add(10,32)

```
Main.test "func add(a,b) = a + b; add(10,32)"
→ Result = 42
```

What we'll trace:

1. Parse.fromString → AST
2. Compiler.compProg → instruction list
3. Asm.asm → int array
4. VM.exec → result printed